

Obtención de requisitos

Proceso de **recoger información** del cliente / usuario:

- Entrevistas, cuestionarios, reuniones.
- Análisis de documentación existente (manuales, webs, sistemas antiguos).
- Observación de cómo trabajan actualmente.

Resultado: primer **borrador de necesidades**

Análisis de requisitos

Consiste en **entender y ordenar** la información recogida:

- Detectar contradicciones.
- Distinguir **necesidades reales** de deseos vagos.
- Separar requisitos:
 - **Funcionales** (qué debe hacer el sistema).
 - **No funcionales** (rendimiento, seguridad, disponibilidad, normativa. . .).
- Revisar la **viabilidad técnica**

Síntesis de requisitos

La síntesis es **resumir y estructurar**:

- Agrupar requisitos por áreas:
 - Red / comunicaciones.
 - Servidores / sistemas.
 - Bases de datos.
 - Seguridad.
 - Monitorización / copias de seguridad / otros.
- Generar una versión **clara y sin repeticiones**, evitando ambigüedades.

Especificación de requisitos

La especificación es el **documento formal de requisitos** (lo que el cliente “firma”).

Incluye normalmente:

- **Requisitos funcionales (RF)** numerados:
 - Ej: RF-1: El sistema debe permitir la autenticación con usuario/contraseña.
- **Requisitos no funcionales (RNF):**
 - Ej: RNF-3: El servicio web debe estar disponible el 99% del tiempo en horario laboral.
 - Restricciones: tecnologías obligatorias, licencias, normativa, etc.
 - Suposiciones: cosas que se dan por hechas (conexión a Internet, hardware mínimo...).

Deben ser **claros, medibles y verificables**.

Priorización de requisitos

Como no se puede hacer todo a la vez, se **priorizan**:

- Clasificación típica:
 - **Must** (imprescindible).
 - **Should** (muy recomendable).
 - **Could** (si hay tiempo).
 - **Won't** (se descarta de momento).
- Criterios:
 - Impacto en el negocio.
 - Coste / tiempo de implantación.
 - Riesgo técnico.
 - Dependencias entre requisitos.

Resultado: lista de requisitos **ordenada por prioridad**, base para la planificación y el diseño.

Resumen: Fase de análisis

Estamos trabajando con una línea recta



A partir de aquí **no** se inventan requisitos nuevos, se trabaja con los ya definidos.

Si se necesita crear algo nuevo, se debe volver a la fase de análisis

Antes de diseñar:

- Verificar que estén **completos y actualizados**.
- Resolver dudas pendientes.
- Marcar claramente qué requisitos **entran en el alcance** del proyecto.

Todo esto se suele resolver con los **stakeholders**

Fase Diseño: Planificación inicial

A partir de los requisitos priorizados:

- Definir **hitos** reales:
 - Entorno montado.
 - Arquitectura definida.
 - Servidores y balanceador configurados.
 - Seguridad básica implantada.
 - Pruebas y entrega.
- Dividir en **tareas**:
 - Instalar X.
 - Configurar Y.
 - ...
- Estimar **duración** y **responsables**.

Herramientas posibles:

- Diagrama de Gantt.
- Lista de tareas (Trello, hoja de cálculo, etc.).

¿Qué es la arquitectura del sistema?

Es la **vista global** de cómo estará montada la solución:

- Qué **componentes** hay (servidores web, DB, proxy, balanceador, firewall. . .).
- Cómo se **conectan** entre sí (redes, VLAN, DMZ. . .).
- Dónde se sitúan los **usuarios** y otros sistemas externos.

Normalmente:

- **Arquitectura lógica:**
 - Capas (presentación, lógica de aplicación, datos).
 - Módulos: web, API, DB, autenticación, etc.
- **Arquitectura física / despliegue:**
 - Máquinas físicas, VM o contenedores.
 - Redes, subredes, routers, balanceadores.

Fase Diseño: Contenido de los esbozos

En esta fase no buscamos todavía un diagrama perfecto, sino un **dibujo claro de “qué hay” y “cómo se conecta”**.

El objetivo es que cualquier persona del equipo pueda entender, de un vistazo, la forma general del sistema.

1. Lista de nodos (máquinas y componentes principales)

Un **nodo** es un elemento que “existe” en la arquitectura y que ejecuta algo o protege algo:

- Servidores:
 - “Servidor Web 1”, “Servidor Web 2”
 - “Servidor de Bases de Datos”
 - “Servidor de Backup”
- Dispositivos de red:
 - “Balanceador de carga”
 - “Router de salida a Internet”
- Otros componentes relevantes:
 - “Servidor LDAP / Directorio”
 - “Proxy inverso”

2. Relaciones entre nodos (quién habla con quién)

Después se dibujan las **conexiones** entre nodos:

- Líneas que unen nodos indicando:
 - Protocolo y puerto:
 - HTTP (80), HTTPS (443), SSH (22), MySQL (3306), etc.
 - Sentido de la comunicación (si es relevante):
 - Por ejemplo, flecha desde “Servidor Web” → “Servidor DB”.
- Ejemplos:
 - Cliente web → Balanceador → Servidor Web 1 / Servidor Web 2.
 - Servidor Web → Servidor DB (puerto 3306/TCP).
 - Servidor de Monitorización → Todos los servidores (puerto 9100/TCP, por ejemplo).

No se trata de poner **todos** los puertos posibles, sino los **principales** que explican cómo funciona el sistema.

3. Zonas de red (segmentación básica)

Las **zonas de red** ayudan a visualizar la seguridad y la organización del sistema.

Las más típicas son:

- **Red interna**
 - Donde están los servidores que solo deberían ser accesibles desde dentro (DB, backup, etc.).
- **DMZ (zona desmilitarizada)**
 - Donde colocamos servicios que reciben tráfico desde Internet (servidores web, proxy inverso, balanceador).
- **Red de administración** (si la hay)
 - Solo para administración remota (SSH, RDP, paneles de gestión).

4. Clientes (actores que usan o administran el sistema)

No todo es “servidores”: también hay que dibujar **quién usa el sistema**:

- **Usuarios internos**

- Personal de la empresa, conectados desde la red corporativa o por VPN.
- Ejemplo: “Técnicos de sistemas”, “Personal de administración”.

- **Usuarios externos**

- Clientes, público general, etc. que acceden desde Internet.

- **Administración / DevOps**

- Personas que gestionan el sistema:
 - Acceso por VPN + SSH.
 - Acceso a paneles web de administración.

Diagramas típicos:

- **Diagrama de contexto:**
 - Sistema en el centro, actores alrededor (usuarios, otros sistemas).
- **Diagrama de componentes:**
 - Módulos lógicos: web, autenticación, DB, servicios externos.
- **Diagrama de despliegue / red:**
 - Servidores, redes, balanceador, firewall, etc.
- **Diagrama de flujo de datos (DFD):**
 - Flujo de datos entre componentes (ej: login, consulta, respuesta).

Fase Diseño: Diagramas

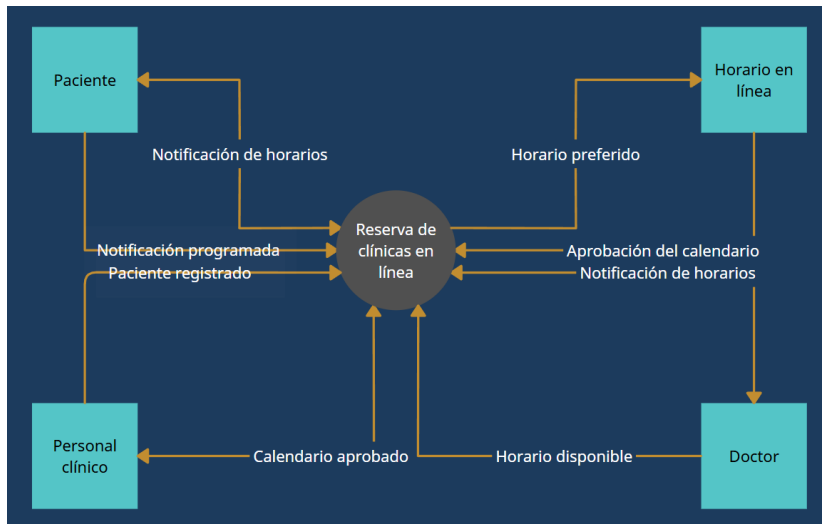


Figura 1: Ejemplo de diagrama de contexto

Fase Diseño: Diagramas

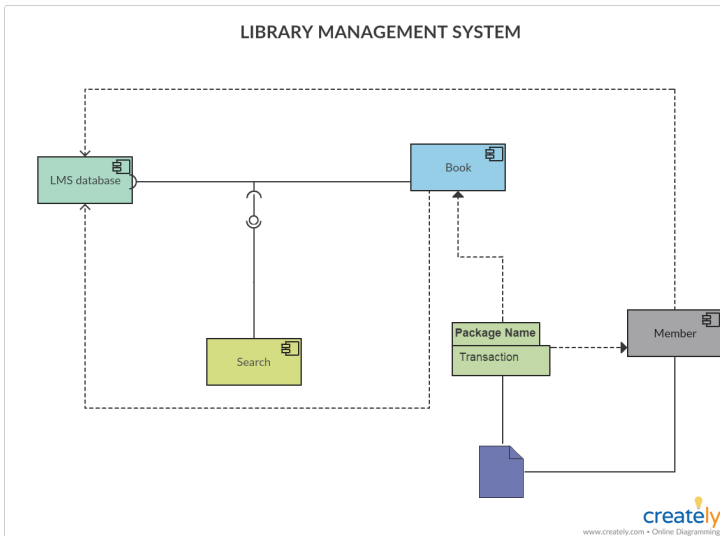


Figura 2: Ejemplo de diagrama de componentes

Fase Diseño: Diagramas

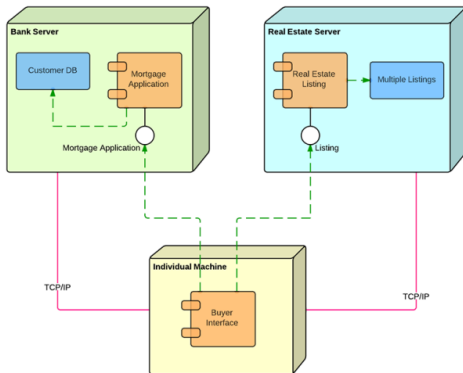


Figura 3: Ejemplo de diagrama de despliegue

Fase Diseño: Diagramas

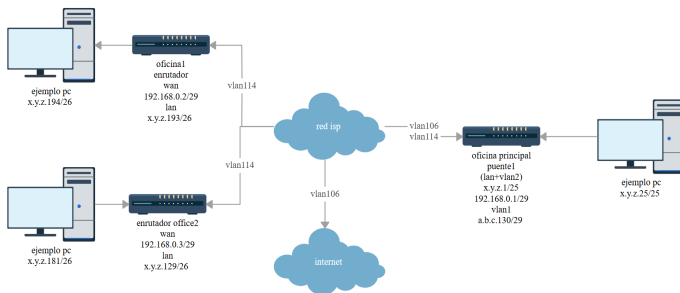


Figura 4: Ejemplo de diagrama de red

Fase Diseño: Diagramas

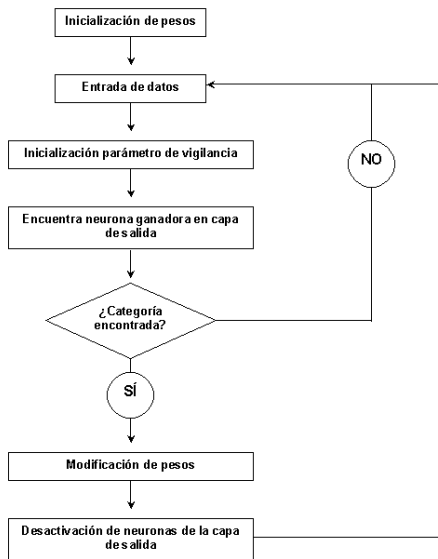


Figura 5: Ejemplo de diagrama de flujo de red

Requisitos de los diagramas

- Deben ser **legibles y coherentes** con la descripción de arquitectura.
- Usar nombres claros de nodos y servicios.
- Guardarlos como **imagen o PDF** dentro de la documentación (ej: export de draw.io).

¿Qué consiste?

Definir **cómo se va a proteger** cada parte del sistema.

Activos a proteger

- Servidores (SO, servicios).
- Bases de datos (datos sensibles).
- Comunicaciones (red interna, DMZ, acceso remoto).
- Cuentas de usuario y credenciales.

Seguridad de red

- Segmentación:
 - Red interna, DMZ, VLANs.
- Firewalls:
 - Puertos abiertos y entre qué redes.
 - Tabla de reglas básica:

Origen	Destino	Puerto/Protocolo	Acción
DMZ	Servidor DB	3306/TCP	Allow
Ext	Red interna	*	Deny

- VPN para acceso remoto seguro (si se necesita).

Seguridad de sistemas

- Política de cuentas:
 - No usar root directamente, uso de sudo.
- Actualizaciones de sistema y servicios.
- Minimización de servicios:
 - Deshabilitar lo que no se use.
- Registros / logs y ubicación.

Seguridad de aplicación

- Uso de **HTTPS** en lugar de HTTP.
- Autenticación y autorización:
 - Quién puede hacer qué.
- Gestión de errores:
 - No mostrar información sensible en mensajes de error.

Seguridad de datos

- Copias de seguridad:
 - Qué se copia, frecuencia, dónde se guarda.
- Cifrado (si aplica).
- Políticas de acceso a la DB (roles y permisos).

Forma de documentar la seguridad

Se puede usar una tabla por componente:

Componente	Riesgos principales	Medidas de seguridad
Servidor web	Ataques HTTP, vulnerabilidades de servicio	HTTPS, actualizaciones, WAF

O bien por apartados:

- Seguridad de red
- Seguridad de sistemas
- Seguridad de aplicación
- Copias de seguridad y recuperación

Decisiones principales

- Tipo de DB:
 - Relacional (MySQL/MariaDB, PostgreSQL, etc.).
- Ubicación:
 - Servidor dedicado, compartido, contenedor. . .
 - Red en la que se encuentra (solo accesible desde red interna/DMZ).
- Acceso de la aplicación:
 - Host, puerto, protocolo.
 - Drivers / librerías.

Usuarios, roles y permisos

- Definir **usuarios de aplicación** (no usar root).
- Roles típicos:
 - Usuario lectura/escritura.
 - Usuario solo lectura (si hace falta).
- Permisos mínimos necesarios:
 - Qué bases de datos / tablas puede usar cada usuario.

Ejemplo de tabla:

Usuario DB	Uso	Permisos	Host de origen
app_user	Aplicación web	SELECT, INSERT...	Servidor web/balancer
backup_user	Copias seguridad	SELECT sobre todas	Servidor backup

Disponibilidad y rendimiento (opcional)

- Replicación o alta disponibilidad de la DB.
- Uso de pool de conexiones.
- Límites básicos de conexiones simultáneas.